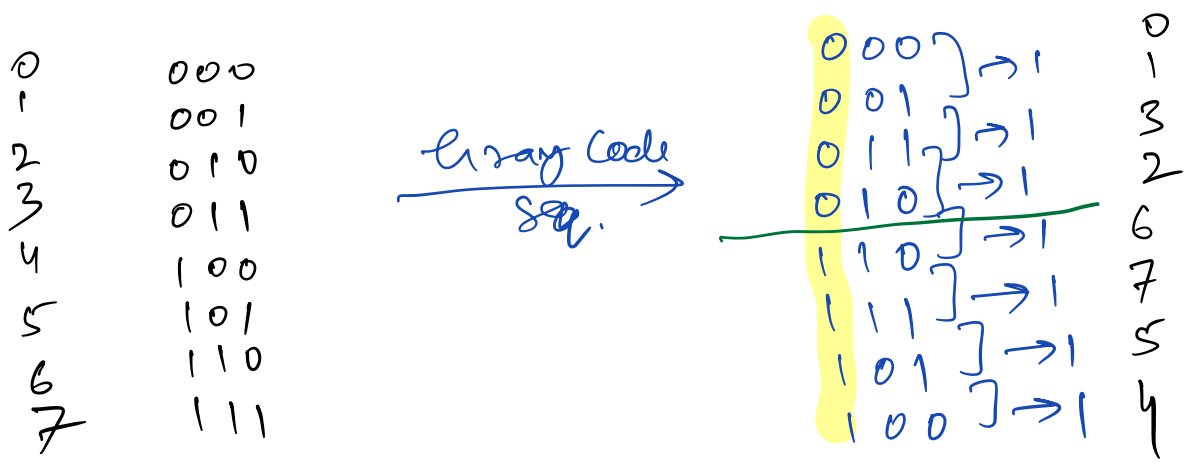
1. Gray Code
2. Valid Sudoku
3. Flatten Nested List
4. Comparator concept

} to be discussed today

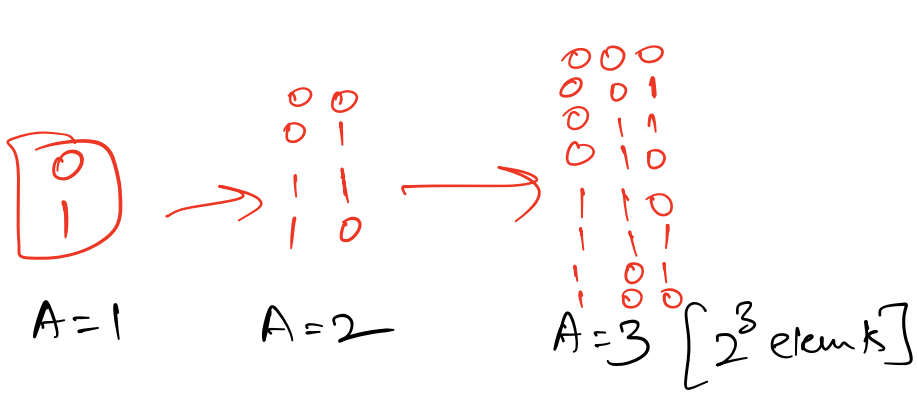
Gray Code

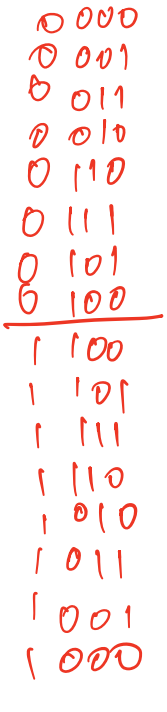
→ binary numeral system where two successive values differ in only one bit.



Question: Given an `A` > 0 return gray code sequence where each element will have A bits.

$1 \le A \le 16$





A=4
[2^4 ele]

Code

```
vector<int> grayCode(int A) {
    vector<int> ans = {0, 1};
    for (int i=2; i<=A; ++i) {
        int currSize = ans.size();
        for (int j = currSize-1; j>=0; --j) {
            ans.push_back(ans[j] + 2^(i-1));
```

(2^(i-1) ⇓ 1<<(i-1))

```
    }
}
return ans;
}
```

ans = {0, 1}

A = 1

ans = {0, 1, 1+2^1, 0+2^1}

A = 2

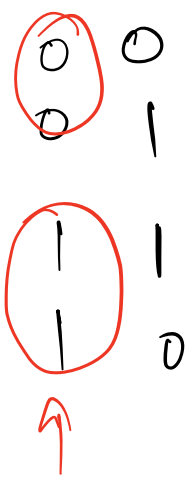
{0, 1, 3, 2}

ans = {0, 1, 3, 2, 2+2^2, 3+2^2, 1+2^2, 0+2^2}

{0, 1, 3, 2, 6, 7, 5, 4}

A = 3

Why code is working ?



1 → 11 (3)

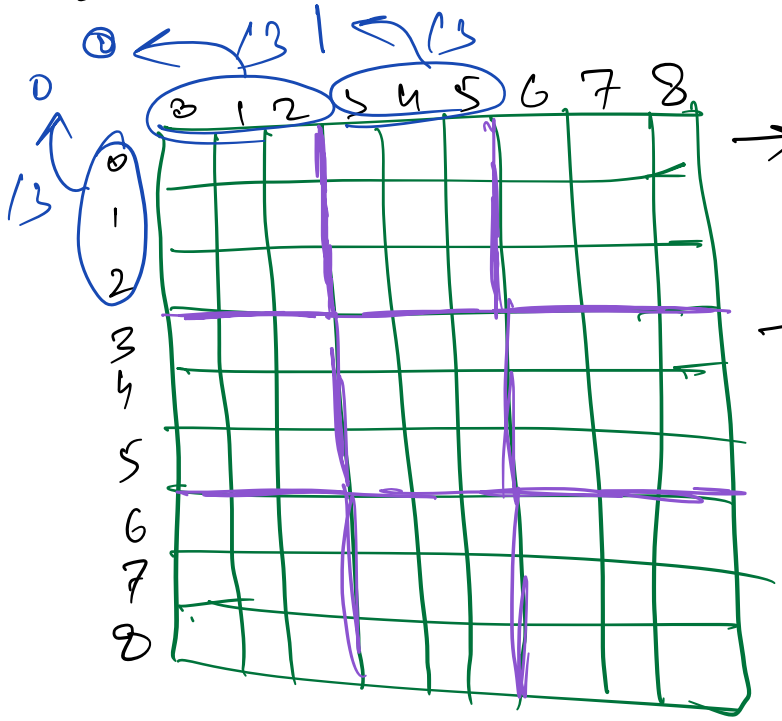
1 → 2 → 3

Time complexity ≈ O(2^A)

Space " ⇒ O(1)

Valid Sudoku

Sudoku rules → 9x9 grid



→ Each row should have 1-9 exactly once

→ Each column same rule

→ Each box same rule

Input format

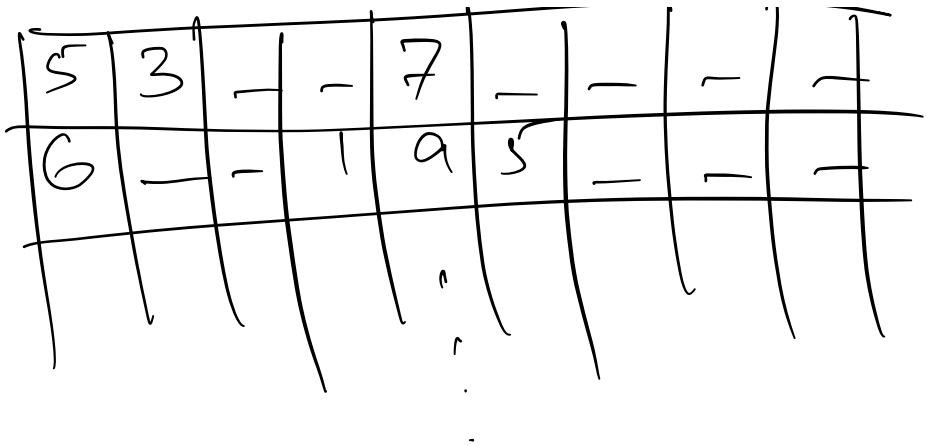
→ Array of 9 strings

→ Each string has 9 characters

→ if a char is ['1', '9'] then it means a number.

→ if a char is '.' it means empty.

eg ["53..7....", "6..195...", ...]



Code

```
bool isValidSudoku(vector<string> board){
    bool r[9][9], c[9][9], s[3][3][9];

    clear(r);
    clear(c);
    clear(s);

    for(int i=0; i<9; ++i){
        for(int j=0; j<9; ++j){
            if(board[i][j] != '.'){
                int n = board[i][j] - '1';
                if(r[i][n] == true)
                    return false;
                r[i][n] = true;
                if(c[j][n] == true)
                    return false;
```

clear fn will mark all val. as false

```
return false;
c[j][n] = true;
if( s[i/3][j/3][n] == true)
    return false;
s[i/3][j/3][n] = true;
}
}
}
return true;
}
```

Explaination for box indices

box 1 $i \to \{0,1,2\}$ $i/3, j/3 \to [0,0]$

$j \to \{0,1,2\}$

box 2 $i \to \{0,1,2\}$ $i/3, j/3 \to [0,1]$

$j \to \{3,4,5\}$

⋮

Resum at 10:15 PM

Flatten Nested List

[2, 4, [41, 56], 5]

[1, 2, [4, [5, [6, 7], 7], 5, 4]]

NestedInteger — already given, is supporting this storage

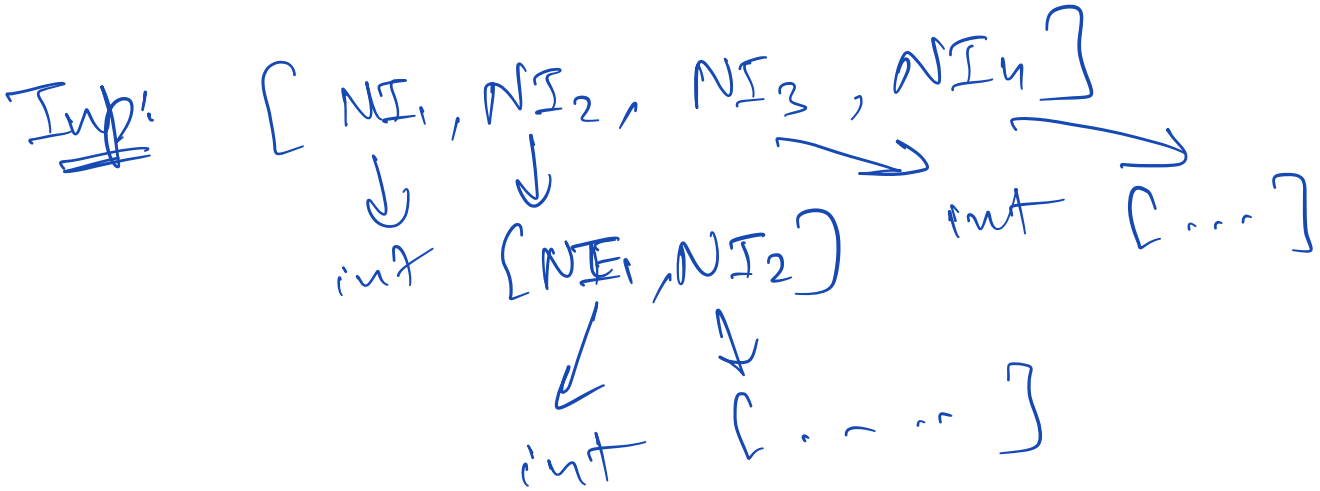
1. isInteger() → bool
2. getInteger() → int
3. getList() → vector<NestedInteger>

Nested Iterator

Implement 3 functions

1. NestedIterator(vector<NestedInteger> list);
2. int next();
3. bool hasNext();

How to iterate ?



[7, 5, [6, [1, 2, [3, 9], 6], 8, 5], 2]

⇓

[7, 5, 6, 1, 2, 3, 9, 6, 8, 5, 2]

Solution

Create a 'ans' vector & flatten the Nested list in constructor;

vector<int> ans;
int index;

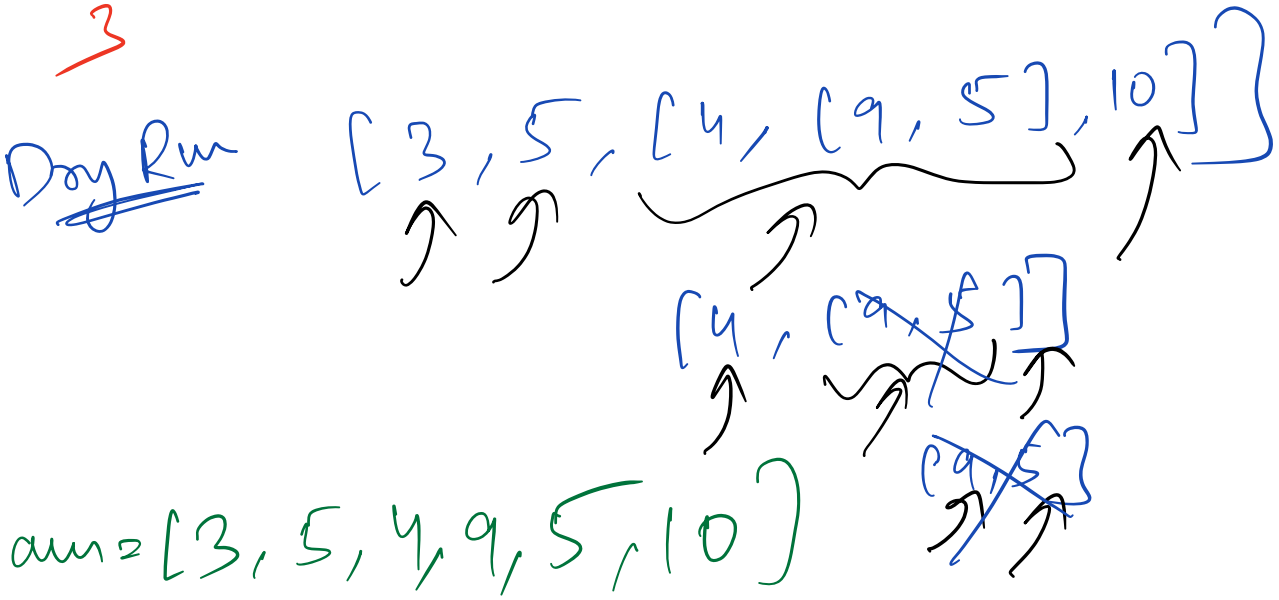
] members of NestedIterator

```
NestedIterator(vector<NestedInteger> list){
    flatten(list);
    int index =0;
}

void flatten(vector<NestedInteger> list){
    for(int i=0; i<list.size(); ++i){
        if(list[i].isInteger()){
            ans.push_back(list[i].getInteger());
        } else {
            flatten(list[i].getList());
        }
    }
}

bool hesNext(){
    if(index < ans.size()){
        return true;
    }
    return false;
}
```

recursion

```
int next () {
    return ans[index++];
}
```

Dry Run



Comparator f^n in Sorting

Sort a list in increasing order

How? → `sort(a.begin(), a.end());`

```
bool compare(int x, int y) {
    if (x<y)
        return true;
    return false
}
```

Sort a list based on number of factors.

a = [2, 11, 14, 8]

2 2 4 4

sort ⟹ [2, 11, 8, 14]

```
Sort(a.begin(), a.end(), compare);

bool compare(int x, int y){
    int fx = factors(x);
    int fy = factors(y);
    if (fx != fy)
        return fx < fy;
    return x <= y;
}
```

3

sort strings based on size

```
vector<string> a;
sort(a.begin(), a.end(),
                compare);
bool compare(string x, string y){
    return x.size() <= y.size();
}
```